

MODULE I: REVIEW OF ITERATIVE STRUCTURES

Understanding Loops and Control Flow

Introduction to Iterative Structures

What are Iterative Structures?

Iterative structures, commonly known as **loops**, are fundamental programming constructs designed to execute a block of code repeatedly.

They enable automation of repetitive tasks and efficient processing of data collections, eliminating the need for redundant code.

Importance of Iteration

Why are Loops Important?

- ✨ **Automation:** Execute tasks multiple times without manual repetition.
- ⚡ **Efficiency:** Streamline code, making it concise and easier to maintain.
- 📋 **Data Processing:** Seamlessly iterate through data structures like lists and strings.
- ⚖️ **Conditional Control:** Perform actions until a specific condition is satisfied.

Python's Loop Constructs

Types of Loops in Python

For Loop



Topic 1: While Loop

Conditional Execution

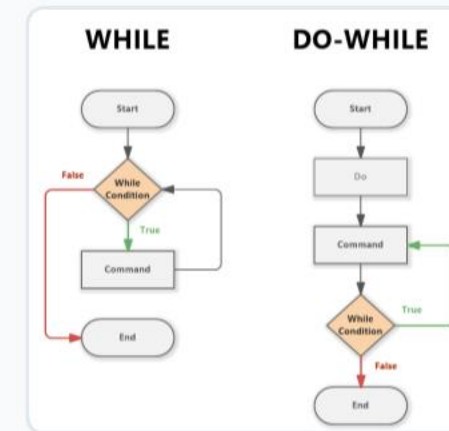
<> Syntax & Basic Usage

```
while condition:  
    # Code block to execute repeatedly  
    # (Ensure condition eventually becomes  
    False)
```

- The while loop executes a block of code **as long as** a specified condition evaluates to True.
- The condition is checked **before** each iteration of the loop.
- It's crucial to include logic that modifies the condition to prevent **infinite loops**.
- This loop is ideal when the number of iterations is **unknown** beforehand.

Logical Flow Diagram

Flowchart of a While Loop



Iteration Scenarios

Practical Examples

1 Counting Example:

```
# Count from 0 to 4  
count = 0
```

Topic 2: Do-while Loop (Emulation in Python)

Concept of the Do-while Loop Emulating Do-while Behavior

Core Definition

A `do-while` loop is an iterative control flow statement that executes its code block ****at least once**** before checking a condition. Subsequent iterations only occur if the condition remains true. This is known as a "post-test" loop.

 Post-Test Logic

Python's Native Absence

Python ****does not have a built-in `do-while` loop****. Unlike languages like C++ Java or JavaScript Python



The while True+ breakPattern

The standard Pythonic way to emulate a do-while loop is with an infinite `while True` loop, executing the logic first and then checking a condition to `break` out.

```
value = 0
while True:
    print(f"Loop ran. Current value: {value}")
    value += 1
    if value > 2: # Condition check at the end
        break
print("Emulated do-while loop finished.")
```

Common Use Cases

 **Input Validation:** Repeatedly prompt a user for input until it's valid, guaranteeing the prompt shows at least

Topic 3: For Loop

<> The for Loop: Iterating Over Sequences

The for loop in Python is for **iterating over a sequence** (like a list, tuple, string) or other iterable objects. It processes each item until the collection is exhausted.

General Syntax:

```
for variable in iterable:  
    # code block to execute for each item
```

The loop automatically manages iteration, assigning each item from the iterable to variable in successive steps.



Using the range() Function

The built-in range() function generates a sequence of numbers, making it easy to iterate a specific number of times.

range(stop)

Generates numbers from 0 up to (not including) stop.

```
# Generates 0, 1, 2, 3, 4  
for i in range(5):  
    print(i)
```

range(start, stop)

Generates numbers from startup to (not including) stop.

```
# Generates 2, 3, 4, 5  
for i in range(2, 6):  
    print(i)
```

range(start, stop, step)

Topic 4: Nested Loops

Understanding Nested Loop Structure



Hierarchical Iteration



Definition: A **nested loop** is where one loop is placed inside another. The 'inner' loop is contained within the body of the 'outer' loop.



Execution Flow: For each single iteration of the outer loop, the entire inner loop completes all of its iterations. The inner loop's body will execute $N * M$ times.



Basic Syntax:

```
# Outer loop
for outer_item in outer_collection:
    # Inner loop
    for inner_item in inner_collection:
        # Code executed N * M times
        print(f"Outer: {outer_item}, Inner: {inner_item}")
```

Example: Matrix Operations (2D List Traversal)



Multi-Dimensional Traversal



Ideal for processing matrices (lists of lists). Outer loop handles rows, inner loop handles elements in each row.

```
matrix = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]
print("Matrix Elements:")
for row in matrix:
    for element in row:
        print(element, end=" ")
    print() # New line after each row
```

Example: Pattern Printing



Generative Patterns



A classic use case where outer loops control rows and inner loops control columns to create character patterns.

```
# Right-Angled Triangle
rows = 5
for i in range(1, rows + 1):
    for j in range(i):
```

Output Preview:

```
*
* *
* * *
* * * *
* * * * *
```

Topic 5: Break and Continue Statements

The `break` Statement: Exiting Loops



🛑 Immediate Exit

➤ Inner Loop Termination

📌 The `break` statement is used to **terminate the current loop prematurely**. When encountered, the loop stops, and execution continues at the statement following the loop.

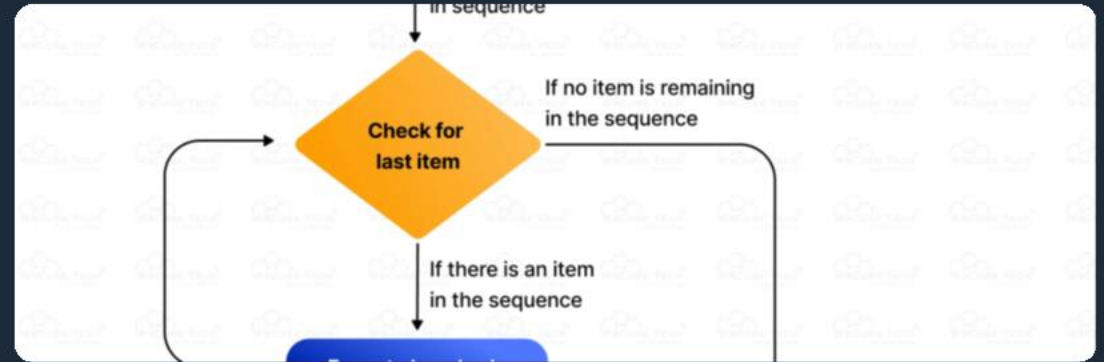
💡 In nested loops, `break` only exits the **innermost loop**.

```
# Example: Find the first number divisible by 7
```

```
for num in range(10, 30):
```

```
    if num % 7 == 0:
```

The `continue` Statement: Skipping Iterations



➤ Skip Current Iteration

⚙️ Conditional Processing

🔄 The `continue` statement is used to **skip the rest of the current iteration** and move directly to the next one.

💡 Ideal for 'filtering' elements within a loop, processing some items while ignoring others based on a condition.

```
# Example: Process only odd numbers
```

```
for num in range(1, 10):
```

```
    if num % 2 == 0:
```

Summary and Next Steps

📖 Core Concepts Recapped

Key Takeaways from Iterative Structures

- ✨ **Loops as Automation:** ``for`` and ``while`` loops automate repetitive tasks and reduce code redundancy.
- ⚖️ **Conditional vs. Definite Iteration:** Use ``while`` for unknown iteration counts and ``for`` for known sequences.
- ↔️ **Flow Control Mastery:** ``break`` provides an immediate exit, while ``continue`` skips to the next iteration.
- 📦 **Nested Loops & Performance:** Powerful for multi-dimensional data, but be mindful of increased time complexity.
- 💡 **Python's Approach:** Emulate a ``do-while`` loop's "execute-at-least-once" behavior with ``while True`` and ``break``.



⚖️ Strategic Loop Selection

Choosing the Right Loop for the Job

- ☰ **Use ``for`` Loop:** For iterating over fixed sequences (lists, ranges) when the number of iterations is pre-determined.
- 🔄 **Use ``while`` Loop:** For repeating actions until a dynamic condition is met, where repetitions are unknown beforehand.
- ▶️ **Emulate ``do-while``:** Use ``while True`` with ``break``



•

•